

Báo cáo kỹ thuật Giai đoạn 3: Huấn luyện Mô hình Học sâu và Kết quả Thực nghiệm (Model & Results)

1. Đặt vấn đề và Mục tiêu huấn luyện (Objective)

Giai đoạn 3 tập trung vào thiết kế kiến trúc, cấu hình siêu tham số, thực thi huấn luyện và đánh giá chi tiết các mô hình học máy thành phần trong pipeline xác thực phương tiện. Mục tiêu là xây dựng và tối ưu hóa các mô hình nhận diện biển số (YOLOv8-nano), trích xuất văn bản biển số (ban đầu EasyOCR, sau chuyển sang **PaddleOCR** — xem §3.2) và các mô hình học sâu phân loại đặc trưng phương tiện (Hãng xe và Màu sắc xe) sử dụng phương pháp Học chuyển vị (Transfer Learning) nhằm tối ưu hóa độ trễ và tài nguyên phần cứng.

2. Nghiên cứu tài liệu tham khảo (Literature Review)

Trong quá trình phát triển mô hình, nhóm đã dựa trên các cơ sở lý thuyết và nghiên cứu thực nghiệm sau: 1. **YOLOv8 Architecture (Jocher et al., 2023 - Ultralytics)**: Nghiên cứu giới thiệu kiến trúc YOLOv8 cải tiến với cấu hình không có neo (anchor-free), nâng cao đáng kể độ chính xác định vị vùng hộp biển số trong khi vẫn duy trì tốc độ suy luận thời gian thực ở mức cao trên phần cứng CPU/GPU yếu. 2. **Deep Residual Learning for Image Recognition (He et al., 2016 - ResNet)**: Công trình đặt nền móng cho cơ chế kết nối tắt (shortcut connections) giúp giải quyết hiện tượng suy giảm gradient trong các mạng nơ-ron rất sâu. Nhóm đã tham khảo cấu trúc ResNet để thiết kế mạng phân loại hãng xe EfficientNet-B0 (vốn sử dụng cơ chế mở rộng Compound Scaling hiệu quả hơn). 3. **Searching for MobileNetV3 (Howard et al., 2019)**: Giới thiệu kiến trúc MobileNetV3 kết hợp công nghệ NAS (Network Architecture Search) và thiết kế NetAdapt, mang lại giải pháp phân loại ảnh vô cùng

gọn nhẹ và tiết kiệm năng lượng cho thiết bị biên. Đây là cơ sở khoa học để nhóm lựa chọn MobileNetV3-Small cho tác vụ phân loại màu sắc xe.

3. Kiến trúc các mô hình thành phần (Model Architectures)

3.1. Bộ phát hiện vị trí biển số (YOLOv8 Plate Detector)

- **Kiến trúc:** YOLOv8-nano (`yolov8n.pt`).
- **Đặc điểm:** Đây là phiên bản nhỏ nhất trong dòng YOLOv8 với số lượng tham số cực kỳ tối ưu, giúp suy luận cực nhanh trên CPU mà không cần card đồ họa chuyên dụng.
- **Đầu ra:** Tọa độ bounding box $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ bao quanh biển số xe.

3.2. Bộ nhận diện ký tự biển số (OCR Engine: PaddleOCR)

- **Lựa chọn engine (Benchmark C):** Ban đầu nhóm dùng **EasyOCR** (ResNet + LSTM + CTC). Tuy nhiên benchmark trên 16 biển CCTV thật cho thấy EasyOCR đọc đúng **0%** chuỗi (exact-match), trong khi **PaddleOCR (CRNN+CTC)** đạt **81%** (CER 0.28 $\rightarrow$ 0.03). Vì vậy **PaddleOCR là engine OCR duy nhất ở runtime**; nếu PaddlePaddle không khả dụng, hệ thống báo lỗi cứng thay vì âm thầm chuyển sang EasyOCR. EasyOCR chỉ còn dùng cho benchmark/đánh giá (train/eval-only). Chi tiết: `docs/benchmarks/ocr_benchmark.md`.
- **Cấu hình:** Chạy ngoại tuyến hoàn toàn; engine cấu hình ở `main/configs/config.yaml` (`ocr.engine: ppocr` — runtime PaddleOCR-only, không còn fallback engine).

3.3. Bộ phân loại hãng xe (Brand Classifier)

- **Backbone:** **EfficientNet-B0** (đã đóng băng các lớp trích xuất đặc trưng tiền huấn luyện trên ImageNet).
- **Tầng phân loại bổ sung:**

- `GlobalAveragePooling2D()`: Làm phẳng bản đồ đặc trưng 2D từ backbone.
- `Dropout(0.5)`: Giảm tỷ lệ khớp quá mức xuống 50% bằng cách ngắt ngẫu nhiên một nửa số nơ-ron kết nối trong mỗi batch huấn luyện.
- `Dense(8, activation="softmax")`: Đầu ra phân lớp cho 8 thương hiệu mục tiêu.
- *Lưu ý pivot*: khác với ResNet50 trong đề xuất ban đầu (Report 1), nhóm chọn **EfficientNet-B0** vì cùng độ chính xác nhưng nhẹ và nhanh hơn nhiều trên CPU (xem Benchmark A màu, cùng kết luận về kích thước/độ trễ).

3.4. Bộ phân loại màu sắc xe (Color Classifier)

- **Backbone: MobileNetV3-Small** (đã đóng băng các lớp trích xuất đặc trưng tiền huấn luyện trên ImageNet).
- **Tầng phân loại bổ sung** (dựng bằng Functional API, gọi `base(x, training=False)` để BatchNorm chạy chế độ inference):
 - `Rescaling(255.0)`: đưa ảnh [0,1] về [0,255] đúng kỳ vọng của MobileNetV3 (`include_preprocessing=True` tự chuẩn hoá nội bộ). *Lưu ý: phiên bản đầu cộng thêm `Rescaling(1/127.5, -1)` gây double-preprocessing — đã loại (xem mục 5.0).*
 - `GlobalAveragePooling2D()`
 - `Dropout(0.3)`: Tỷ lệ dropout 30% phù hợp với mạng nhỏ.
 - `Dense(8, activation="softmax")`: Phân loại 8 nhóm màu xe.

4. Quá trình huấn luyện và Cấu hình siêu tham số (Training Configuration)

Quá trình huấn luyện được thực hiện cục bộ trên hệ điều hành macOS sử dụng CPU thông qua script [train.py](#).

4.0. Cấu hình cơ sở

- **Hai pha huấn luyện:** (1) **head đông cứng** — backbone freeze, chỉ train tầng Dense, Adam lr $=10^{-3}$; (2) **fine-tuning** — mở băng nửa trên backbone (BatchNorm vẫn freeze), Adam lr $=10^{-4}$.
- **Hàm mất mát:** Categorical Crossentropy (phân loại đa lớp).
- **Batch Size:** 32 mẫu/batch.
- **Số lượng Epoch:** tối đa 15/pha với `EarlyStopping` (môi trường training cô lập TF/Keras, tách khỏi runtime PaddleOCR).
- **Cơ chế giám sát nâng cao:**
 - `EarlyStopping`: Tự động dừng huấn luyện nếu giá trị `val_loss` không cải thiện liên tiếp sau 5 epochs, đồng thời khôi phục trọng số tốt nhất trước đó.
 - `ModelCheckpoint`: Tự động lưu mô hình có giá trị `val_loss` thấp nhất dưới dạng file `.keras`.

4.1. Tinh chỉnh siêu tham số — quá trình và kết quả (Hyperparameter Tuning)

Nhóm **không** dùng công cụ tự động hoá dạng Keras Tuner/Optuna cho các mô hình phân loại (lý do nêu ở cuối mục này), mà thực hiện **quét thủ công có hệ thống** trên từng trục siêu tham số, có đo lại kết quả sau mỗi lần đổi cấu hình. Bốn trục đã quét:

(a) Learning rate (pha head đông cứng): lần chạy đầu dùng lr $=10^{-4}$ cho pha head — quá thấp khiến `EarlyStopping` dừng sớm trước khi mô hình hội tụ (xem §5.0). Sau khi quan sát hiện tượng này, nhóm nâng lr pha head lên 10^{-3} ; lr pha fine-tune giữ nguyên ở 10^{-4} (mở băng một phần backbone nên cần bước học nhỏ hơn để không phá vỡ trọng số tiền huấn luyện). Cấu hình $10^{-3}/10^{-4}$ được chốt làm cấu hình cơ sở (§4.0).

(b) Protocol freeze/unfreeze hai pha: đã mô tả ở §4.0 — head đông cứng trước, sau đó mở băng nửa trên backbone (giữ BatchNorm ở chế độ inference, xem lỗi kỹ thuật #2 ở §5.0). Đây là trục có tác động lớn nhất đến model màu (xem bảng bên dưới).

(c) Các đòn bẩy của model màu (data, fine-tune scope, class-weight, label smoothing, TTA) — quét tuần tự, mỗi bước đo lại trên tập test giữ-riêng (số liệu đối

chiếu với docs/benchmarks/color_finetune_report.md và bảng "Hành trình cải tiến" ở §5.1):

Cấu hình thử	Test accuracy	Macro-F1
Head đông cứng (frozen, data cũ, 783 ảnh)	48.3%	0.48
Fine-tuning nửa backbone (data cũ, 783 ảnh)	55.1%	0.545
Full fine-tune + bổ sung dữ liệu VCoR 600 ảnh/lớp (chưa TTA)	77.6%	0.776
Full fine-tune + VCoR full (5.881 ảnh) + class-weight + label-smoothing 0.1 (plain)	85.3%	0.829
+ Test-time augmentation (TTA, hflip) — DEPLOYED	86.3%	0.841

Các dòng là các bước quét **tuần tự** (mỗi bước thêm một nhóm thay đổi so với bước trước): mở rộng scope fine-tune (frozen → full backbone) cùng với đổi dữ liệu sang VCoR đóng góp mức tăng lớn nhất (48.3% → 77.6%); bước tiếp theo gộp ba thay đổi — mở rộng VCoR từ 600 ảnh/lớp lên full 5.881 ảnh + class-weight + label-smoothing — cộng thêm ~7,7 điểm (77.6% → 85.3%), do đo chung trong một bước nên không tách được đóng góp riêng của từng yếu tố; TTA cộng thêm ~1 điểm cuối (85.3% → 86.3%).

(d) Ngưỡng quyết định (decision threshold) ở tầng cross-check màu: tách biệt với hyperparameter huấn luyện mô hình, nhóm cũng quét ngưỡng tin cậy `color_warn_conf` dùng để gate cảnh báo tráo biến — quét từ 0,00 đến 0,60 và chọn **0,40** làm điểm vận hành (đánh đổi false-alarm ↔ detection). Bảng quét đầy đủ và lý do chọn 0,40 đã trình bày ở Report 4 §4.3(b) (docs/benchmarks/security_eval.md), không lặp lại số liệu ở đây.

Vì sao không dùng Keras Tuner/Optuna: hai lý do kỹ thuật/thực tế buộc nhóm chọn quét thủ công có hệ thống thay vì search tự động. Thứ nhất, môi trường training TF/Keras phải **cô lập hoàn toàn** khỏi runtime PaddleOCR (xung đột thư viện đã ghi ở §3.2/§5.0) — một vòng lặp Tuner/Optuna gọi lại pipeline training nhiều lần trong cùng process sẽ làm tăng rủi ro xung đột và độ phức tạp vận hành trên máy cục bộ. Thứ hai, ngân sách compute (CPU cục bộ cho phần lớn huấn luyện + Colab miễn phí cho phần fine-tune màu) không đủ để chạy hàng chục đến hàng trăm trial tự động như search-based tuning thường cần. Bù lại, các trục nhay cảm nhất với kết

quả cuối — learning rate, mức độ unfreeze backbone, chất lượng/khối lượng dữ liệu, class-weight/label-smoothing/TTA, và ngưỡng quyết định ở tầng cross-check — đều đã được quét thủ công **có đo lại số liệu cho từng bước**, nên dù không dùng công cụ tự động, quá trình vẫn có tính hệ thống và có thể tái lập.

5. Kết quả thực nghiệm và Biểu đồ huấn luyện (Experimental Results)

5.0. Hai lỗi kỹ thuật ban đầu và cách khắc phục

Những lần huấn luyện Keras **đầu tiên** cho kết quả gần như ngẫu nhiên (hãng ~29%, màu ~14%). Khi chuẩn hoá lại pipeline về TF/Keras, nhóm xác định nguyên nhân **không phải** do domain gap mà do **hai lỗi dựng mô hình**: 1. **Double-preprocessing**: lớp `Rescaling(1/127.5, -1)` được chèn *trên đầu* lớp tiền xử lý nội bộ của MobileNetV3 → sai dài giá trị, accuracy rơi về ~1/8. Đã bỏ, chỉ giữ `Rescaling(255.0)` cho cả hai backbone. 2. **BatchNorm chạy ở chế độ training khi backbone đông cứng**: dựng bằng `Sequential` khiến BatchNorm của backbone dùng thống kê từng batch, phá vỡ đặc trưng đông cứng → mô hình không học. Đã chuyển sang **Functional API** với `base(x, training=False)`. Ngoài ra `learning_rate` ban đầu (1e-4) quá thấp khiến EarlyStopping dừng sớm; nâng lên **1e-3** cho pha head đông cứng.

Sau khi sửa, đánh giá được thực hiện trên **tập test giữ-riêng** (118 ảnh màu / 119 ảnh hãng, split 70/15/15 — xem Report 2), chưa từng thấy khi huấn luyện.

5.1. Mô hình phân loại màu sắc xe (Color Classifier — MobileNetV3-Small)

Lưu ý runtime: Mô hình TF/Keras (`color_classifier.keras`) chỉ dùng cho **training và đánh giá** ở giai đoạn đầu. Mô hình **runtime/inference hiện tại (đã deploy)** là **PyTorch MobileNetV3-Small** (`main/data/models/color_MobileNetV3Small.pt`, được nạp bởi `main/src/models/torch_color.py`), fine-tune trên Google Colab (GPU) bằng `main/scripts/colab_train_color.py`. Kết quả đánh giá dưới đây được đo lại

(tái lập, không phải số Colab tự báo) trên tập test giữ-riêng bằng script `main/scripts/eval_color_deployed.py`, chạy trực tiếp trên file `.pt` đang chạy ở runtime.

Hành trình cải tiến (journey): ~55% (frozen-backbone, data cũ, 783 ảnh project) → ~78% (full fine-tune + bổ sung dữ liệu VCoR, 600 ảnh/lớp) → **~86% (deploy)** nhờ full VCoR (5,881 ảnh hợp lệ) + ba lever bổ sung: class-weighted loss, label smoothing 0.1, test-time augmentation (TTA, hflip).

Cấu hình	Test accuracy	Macro-F1
Head đông cứng (frozen, data cũ)	48.3%	0.48
Fine-tuning nửa backbone (data cũ, 783 ảnh)	55.1%	0.545
Full fine-tune + VCoR 600/lớp (chưa TTA)	77.6%	0.776
Full fine-tune + VCoR full (5,881 ảnh) + class-weight + label-smoothing — plain	85.3%	0.829
+ Test-time augmentation (TTA, hflip) — DEPLOYED	86.3%	0.841

Methodology: dataset = **VCoR** (Kaggle, Vehicle Color Recognition — ~5,881 ảnh dùng được trên 8 lớp màu, gộp cùng ảnh gốc của nhóm) + full fine-tune toàn bộ backbone MobileNetV3-Small (không đông cứng) + class-weighted loss (bù lệch lớp Silver/Grey hiếm) + label smoothing 0.1 + test-time augmentation (trung bình softmax ảnh gốc + ảnh lật ngang) + body-crop tiền xử lý (bỏ 20% trên/15% dưới ảnh — loại trời/đường). Đánh giá trên tập test giữ-riêng (held-out), stratified split 70/15/15, seed=42 — **889 ảnh test**, chưa từng thấy khi huấn luyện. Mô hình màu vẫn được dùng làm **cảnh báo mềm** trong runtime (xem Report 4), không từ chối cứng.

Lưu ý trung thực (domain gap): 86% là đo trên VCoR — ảnh xe chụp rõ, nền sạch, ánh sáng tốt (nguồn web/marketplace), **không phải ảnh CCTV bãi xe thật**. Hiệu năng khi triển khai trên camera giám sát thực tế nhiều khả năng **thấp hơn** do khác biệt domain (ánh sáng yếu/ngược sáng, nhiễu, góc nghiêng, độ

phân giải thấp). Để bền hơn khi triển khai thật cần bổ sung tiền xử lý white-balance và một lượng nhỏ dữ liệu CCTV thật để fine-tune thêm.

- **Script đánh giá tái lập:** `main/scripts/eval_color_deployed.py` (tái sử dụng hàm load/split/eval từ `main/scripts/colab_train_color.py`).
- **Báo cáo đầy đủ (JSON + Markdown):**
`docs/benchmarks/color_finetune_report.json`,
`docs/benchmarks/color_finetune_report.md`.
- **Biểu đồ huấn luyện (giai đoạn đầu, TF/Keras, data cũ):**
[color_classifier_training_curves.png](#)

Ma trận nhầm lẫn & hiệu năng theo lớp (Colour Classifier — model deployed, tập test VCoR 889 ảnh, TTA)

Lớp	Precision	Recall	Support
Black	0.840	0.773	88
Blue	0.955	0.924	159
Brown	0.980	0.803	122
Grey	0.597	0.763	93
Red	0.978	0.993	137
Silver	0.632	0.615	78
White	0.808	0.874	87
Yellow	0.976	0.984	125
Macro avg	0.846	0.841	889

Ma trận nhầm lẫn (hàng = nhãn thật, cột = nhãn dự đoán, dự đoán TTA):

	Black	Blue	Brown	Grey	Red	Silver	White	Yellow
Black	68	4	0	15	0	1	0	0
Blue	4	147	0	3	0	3	1	1
Brown	5	1	98	12	2	1	1	2

	Black	Blue	Brown	Grey	Red	Silver	White	Yellow
Grey	3	1	1	71	1	14	2	0
Red	0	0	1	0	136	0	0	0
Silver	0	1	0	15	0	48	14	0
White	1	0	0	1	0	9	76	0
Yellow	0	0	0	2	0	0	0	123

Nhận xét: Red/Blue/Yellow/Brown rất mạnh ($F1 \geq 0.88$), màu sắc nổi bật và ít nhập nhằng. Grey và Silver vẫn là cặp khó nhất (Grey recall 0.763 nhưng precision chỉ 0.597 — bị Black và Silver lẫn vào; Silver precision/recall ~0.62) — ba màu trung tính (Black/Grey/Silver/White) là cụm nhập nhằng cố hữu về sắc tố, đúng như dự đoán trong các phiên thử nghiệm trước. Kết quả này củng cố vai trò "cảnh báo mềm" — không dùng màu để từ chối cứng.

5.2. Mô hình phân loại hãng xe (Brand Classifier — EfficientNet-B0, diagnostic only — đã loại khỏi quyết định)

Cấu hình	Test accuracy	Macro-F1
Head đông cứng	32.8%	0.32
+ Fine-tuning	35.3%	0.34

Dù đã sửa lỗi + fine-tune, phân loại hãng vẫn yếu (~35%) — bài toán 8 hãng nhìn từ phía sau với ~70 ảnh/lớp là khó. Kết quả này **củng cố quyết định bỏ phân loại hãng** khỏi cơ chế quyết định.

Hiệu năng theo lớp (Brand Classifier — model fine-tuned, tập test giữ-riêng 119 ảnh)

Nguồn: `main/data/models/brand_classifier_test_report.json`. File này lưu accuracy tổng, macro-F1 và precision/recall/F1/support **theo từng lớp**; không lưu ma trận nhầm lẫn dạng đếm ô (cell-count) như ở mục màu §5.1, nên bảng dưới trình bày theo đúng cấu trúc có sẵn (hàng = nhãn thật, không có cột dự đoán chi tiết theo cặp).

Lớp	Precision	Recall	F1	Support
Ford	0.600	0.200	0.300	15
Honda	0.238	0.333	0.278	15
Hyundai	0.538	0.467	0.500	15
Kia	0.333	0.133	0.190	15
Mazda	0.250	0.133	0.174	15
Mitsubishi	0.286	0.400	0.333	15
Toyota	0.500	0.429	0.462	14
VinFast	0.333	0.733	0.458	15
Macro avg	0.385	0.354	0.337	119

Nhận xét (suy ra từ precision/recall theo lớp, không phải từ ma trận đếm ô — JSON gốc không lưu cặp nhằm cụ thể): Kia và Mazda là hai lớp yếu nhất (F1 chỉ 0.19 và 0.17, recall dưới 0.15 — phần lớn ảnh của hai hãng này bị model gán nhầm sang hãng khác). VinFast có recall rất cao (0.733) nhưng precision thấp (0.333) — model có xu hướng "đổ dồn" nhiều dự đoán về lớp VinFast, kéo theo nhiều ảnh của các hãng khác (đặc biệt Honda, Mazda — cũng có precision thấp) bị phân loại nhầm thành VinFast. Honda cùng cảnh: precision 0.238 rất thấp dù recall 0.333 tạm ổn, cho thấy nhiều ảnh của các hãng khác lại bị đoán thành Honda. Nhìn tổng thể, độ lệch lớn và không nhất quán giữa precision/recall của từng lớp (không có hãng nào đạt cả hai chỉ số cao) củng cố thêm cho quyết định đã nêu ở §5.2: bài toán phân loại 8 hãng xe chỉ từ ảnh đuôi xe với ~70 ảnh/lớp là quá khó để dùng làm căn cứ quyết định, và nhóm đã đúng khi loại brand khỏi cơ chế quyết định (giữ lại thuần thử nghiệm/chẩn đoán).

- Biểu đồ huấn luyện: [brand_classifier_training_curves.png](#)

5.3. Bộ phát hiện biển số (Plate Detector — YOLOv8-nano, Benchmark B)

Tập validation: **1.765 ảnh** (HuggingFace `keremberke/license-plate-object-detection`, một lớp `license_plate`, có chứa biển số xe Việt Nam). Hai cấu hình cùng huấn luyện 80 epoch trên Apple M1 Max (MPS), imgsz 640, batch 16; latency đo trên CPU.

Mô hình	Khởi tạo	Precision	Recall	mAP50	mAP50-95	Latency (ms/ảnh, CPU)	Kích thước (MB)
plate_finetune	transfer từ COCO yolov8n.pt	0.9823	0.9674	0.9896	0.7040	110.3	6.24
plate_scratch	random init (yolov8n.yaml)	0.9816	0.9560	0.9790	0.6972	110.4	6.24

Precision/Recall lấy từ dòng checkpoint tốt nhất trong `results.csv` của mỗi run — tức dòng có `metrics/mAP50(B)` khớp giá trị mAP50 báo cáo ở trên: epoch 60/80 cho `plate_finetune` (`main/data/models/plate_runs/plate_finetune/results.csv`), epoch 70/80 cho `plate_scratch` (`main/data/models/plate_runs/plate_scratch/results.csv`).

Kết luận: `plate_finetune` (transfer learning từ COCO) thắng `plate_scratch` ở mọi chỉ số chính xác (mAP50 +1.06 điểm, mAP50-95 +0.68 điểm) với cùng latency và kích thước, đồng thời hội tụ sớm hơn nhiều (mAP50 $\approx$ 0.97 đã đạt từ epoch 9). Mô hình `plate_finetune` được chọn và xuất sang `plate_yolov8n.onnx` làm bộ phát hiện biển số ở giai đoạn 2 của pipeline.

Chi tiết đầy đủ: `docs/benchmarks/plate_benchmark.md`.

5.4. Bộ nhận diện ký tự biển số (OCR — Benchmark C)

Tập đánh giá: **16 ảnh crop biển số CCTV thật** (gán nhãn tay, gồm 1 biển xe máy 2 dòng). CER = khoảng cách Levenshtein / độ dài ground-truth; latency đo trên CPU.

Phương pháp	Exact-match	Mean CER	Latency (ms/biển)
EasyOCR	0,000	0,278	31,7
EasyOCR + enhance (deskew/CLAHE)	0,062	0,289	47,2
PaddleOCR (ppocr)	0,812	0,031	423,2
PaddleOCR + enhance	0,438	0,092	423,3

Kết luận: PaddleOCR thắng tuyệt đối — exact-match từ 0% (EasyOCR) lên 81,2%, CER giảm từ 0,278 xuống 0,031. Tiền xử lý (deskew + CLAHE + upscale) không giúp ích — nhúc nhích EasyOCR (0 → 6%) nhưng lại gây hại PaddleOCR (0,81 → 0,44), nên bị loại. PaddleOCR chậm hơn EasyOCR ~13x (423 ms vs 32 ms/biển) nhưng vẫn chấp nhận được vì OCR chỉ chạy một lần/xe qua cơ chế parking-trigger, không chạy theo từng khung hình. Lựa chọn: PaddleOCR (ocr.engine: ppocr) là engine OCR duy nhất ở runtime; nếu PaddlePaddle không khả dụng, hệ thống báo lỗi cứng thay vì âm thầm chuyển sang EasyOCR. EasyOCR chỉ còn dùng cho benchmark/đánh giá (train/eval-only).

Chi tiết đầy đủ: docs/benchmarks/ocr_benchmark.md .

5.5. Cơ chế quyết định delivered (plate-primary)

1. **Biển số (PaddleOCR) là khoá chính** — Benchmark C: exact-match 81% so với EasyOCR 0% trên biển CCTV thật.
2. **Màu là "cảnh báo mềm"**: màu lệch so với CSDL chỉ phát cảnh báo (ALLOW_WARN) chống tráo biển, không từ chối cứng.
3. **Bỏ phân loại hăng** khỏi quyết định (giữ lại như thử nghiệm). Đây là cơ chế **delivered** của hệ thống (xem Report 4).

6. Đề xuất cải tiến độ chính xác (Future Enhancements)

1. **Thu thập dữ liệu in-domain**: quay và cắt ảnh xe trực tiếp tại bãi giữ xe Việt Nam với góc máy cố định để khớp phân phối train/test — đây là hướng còn lại quan trọng nhất, vì model màu hiện đo 86% trên VCoR (ảnh web sạch) chứ chưa có số đo trên CCTV thật.
2. **Bộ phân loại màu đã đạt mục tiêu (~86% trên VCoR, xem §5.1)** nhờ full fine-tune + dữ liệu VCoR + class-weight/label-smoothing/TTA; hướng tiếp theo không phải "fine-tune sâu hơn" mà là **thu hẹp domain gap**: thêm white-balance tiền xử lý + một lượng nhỏ dữ liệu CCTV thật để fine-tune tiếp, và xử lý riêng cụm nhập nhằng Grey/Silver/Black/White (vẫn là điểm yếu nhất, xem confusion matrix §5.1).

3. **Cân bằng & mở rộng dữ liệu hãng xe (Brand):** nâng các lớp hiếm (Brown/Yellow tương ứng phía màu, hoặc các hãng ít ảnh phía brand) vượt ~100 ảnh và bổ sung biến thể ánh sáng bãi xe.